



Basketball Simulator

Software Development Document

Author: Yahya Zahya

Repository: github.com/zahyaa/NBASIM_Project_Full

Version: 1.0

Date: April 17, 2026

Table of Contents

1. [Project Overview](#)
2. [Tech Stack & Services](#)
3. [System Architecture](#)
4. [File & Directory Structure](#)
5. [Backend — Code Walkthrough](#)
6. [Frontend — Code Walkthrough](#)
7. [API Endpoint Reference](#)
8. [How to Run Locally](#)
9. [How to Deploy to Production](#)
10. [Environment Variables](#)

1. Project Overview

Basketball Simulator is a full-stack web application that lets users draft real NBA players, build a fantasy roster, and simulate basketball games with an animated play-by-play gamecast. The application pulls live player data from the balldontlie.io NBA API, generates player ratings, and runs a statistical game simulation engine on the server.

Core User Flow

Register / Login → Select Conference & League → Draft 5–12 Players → Pick Opponent Team
→ Simulate Game → Watch Animated GameCast → View W/L Record

2. Tech Stack & Services

2.1 Runtime & Languages

Technology	Role	Version
Node.js	Server runtime	18+ (tested on 25.9.0)
JavaScript (ES6+)	All application code	—
React	Frontend UI library	18.x

2.2 Backend Stack

Package	Purpose
<code>express</code>	HTTP server and REST API routing
<code>mongoose</code>	MongoDB ODM — schema definitions, queries, validation
<code>jsonwebtoken</code>	JWT token generation and verification for auth
<code>bcryptjs</code>	Password hashing (12-round bcrypt)
<code>axios</code>	HTTP client for proxying requests to the NBA API
<code>cors</code>	Cross-Origin Resource Sharing headers
<code>dotenv</code>	Loads <code>.env</code> file into <code>process.env</code>

2.3 Frontend Stack

Package	Purpose
<code>react</code> / <code>react-dom</code>	Component-based UI rendering
<code>react-router-dom</code> v6	Client-side routing (login, draft, game pages)
<code>react-scripts</code> 3.x	Build tooling (Webpack, Babel, dev server with proxy)

2.4 Database

Service	Purpose
---------	---------

MongoDB Community 8.x	Document database storing users, teams, rosters, game history
-----------------------	---

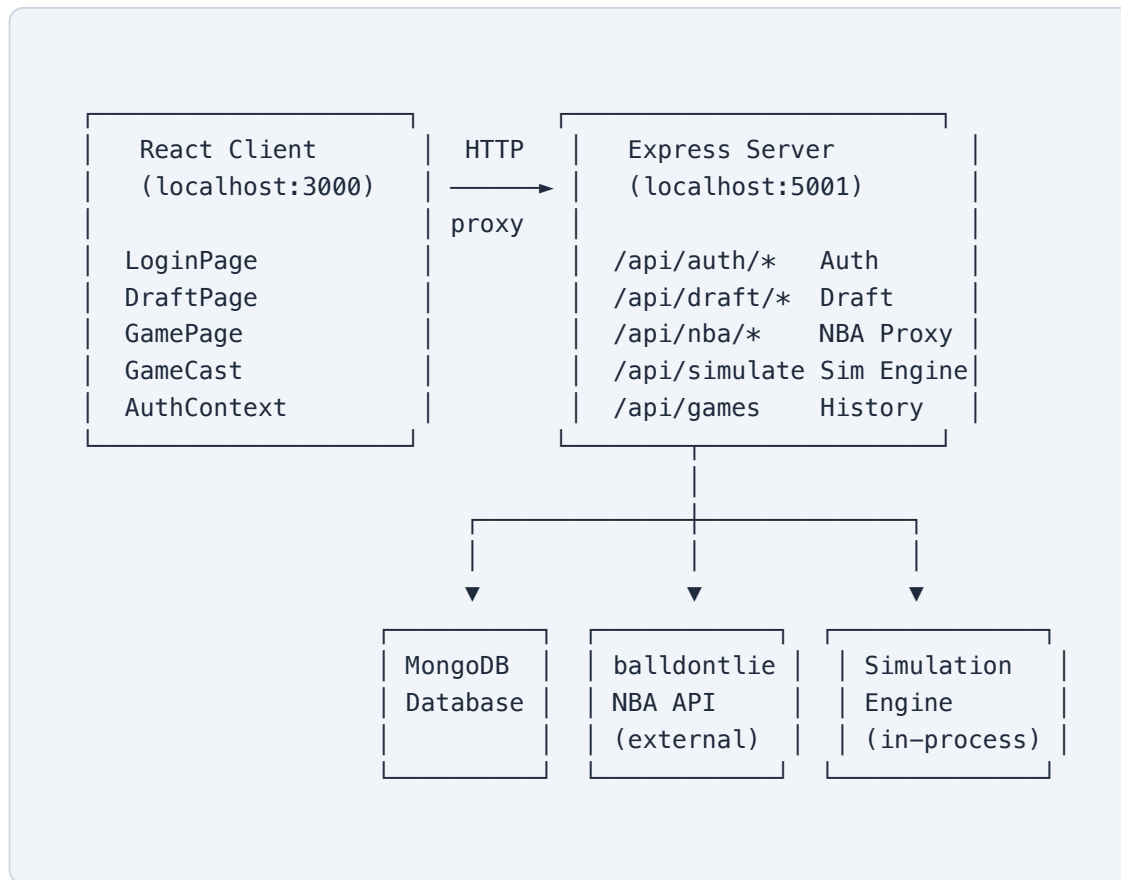
MongoDB Atlas (optional)	Cloud-hosted MongoDB for production deployments
--------------------------	---

2.5 External API

Service	URL	Tier	Data Provided
balldontlie.io	api.balldontlie.io/v1	Free	Teams, Players, Games
		Paid (\$9.99+)	Season Averages, Stats, Box Scores, Standings

Authentication uses an API key passed as an `Authorization` HTTP header on every request.

3. System Architecture



Request flow: The React client makes `fetch()` calls to `/api/*`. In development, react-scripts' built-in proxy forwards these to `http://localhost:5001`. The Express server handles authentication, proxies NBA data requests to `balldontlie.io`, runs the simulation engine in-process, and persists data to MongoDB.

4. File & Directory Structure

```

NBASIM_Project_Full/
├── .gitignore           # Ignores node_modules/, .env, .DS_Store, b
├── README.md           # Project documentation
├── package.json         # Root package (metadata only)
├── server/             # — Express Backend —
│   ├── server.js       # Entry point – wires all middleware & rout
│   ├── .env            # Environment secrets (not committed)
│   ├── .env.example    # Template showing required env vars
│   ├── package.json    # Backend dependencies
│   ├── models/
│   │   ├── User.js     # User schema (auth, team, draft, W/L)
│   │   └── Game.js     # Game schema (teams, scores, play log)
│   ├── middleware/
│   │   └── auth.js     # JWT verification middleware
│   ├── routes/
│   │   ├── auth.js     # POST /register, POST /login, GET /me
│   │   ├── draft.js    # POST /setup, GET /pool, POST /pick, POST
│   │   ├── nba.js      # GET /teams, GET /players, GET /roster (AP
│   │   ├── simulate.js # POST / – run game simulation
│   │   └── games.js    # GET / – game history, POST /save
│   └── services/
│       ├── playerRating.js # Rating algorithms (stats-based & profile-
│       └── simulation.js   # 4Q + OT game engine with play-by-play out
├── client/             # — React Frontend —
│   ├── package.json    # Frontend dependencies + proxy config
│   ├── public/
│   │   └── index.html  # HTML shell
│   └── src/
│       ├── index.js    # React DOM render entry point
│       ├── App.jsx     # Router, nav bar, route guards
│       ├── context/
│       │   └── AuthContext.jsx # Auth state (token, user, login/logout)
│       ├── pages/
│       │   ├── LoginPage.jsx # Login / register form
│       │   ├── DraftPage.jsx # Conference selection + fantasy draft UI
│       │   └── GamePage.jsx  # Opponent selection + simulation trigger

```

```
├── components/
│   ├── GameCast.jsx    # Animated play-by-play with scoreboard
│   ├── GameHistory.jsx # Historical game list
│   └── ...              # Legacy components (Controls, Quarter, etc
```

5. Backend — Code Walkthrough

5.1 `server/server.js` — Entry Point

The main file bootstraps the application:

- Loads environment variables with `dotenv`
- Connects to MongoDB via Mongoose
- Registers middleware: `cors()` and `express.json()`
- Mounts all route modules under `/api/*` prefixes
- Listens on port 5001 (configurable via `PORT` env var)

5.2 `server/models/User.js` — User Schema

Defines the Mongoose schema for user accounts. Key fields:

Field	Type	Purpose
<code>username</code>	String (unique)	Login identifier
<code>password</code>	String	Bcrypt-hashed password (12 salt rounds)
<code>team.name</code>	String	User's custom team name
<code>team.players[]</code>	Array	Drafted roster (playerId, name, position, rating, stats)
<code>conference</code>	String	"East" or "West"
<code>league</code>	String	"NBA", "G-League", or "EuroLeague"
<code>draftCompleted</code>	Boolean	Gates access to the game page
<code>wins / losses</code>	Number	Running game record
<code>gamesPlayed[]</code>	ObjectId refs	References to Game documents

A `pre('save')` hook hashes passwords before persistence. The `toJSON()` method strips the password from API responses.

5.3 `server/models/Game.js` — Game Schema

Stores completed game results: team names, final scores, and the full play-by-play history array. Timestamped automatically.

5.4 `server/middleware/auth.js` — JWT Middleware

Extracts the `Bearer` token from the `Authorization` header, verifies it with `jsonwebtoken`, and attaches `req.userId` for downstream route handlers. Returns 401 on missing or invalid tokens.

5.5 `server/routes/auth.js` — Authentication Routes

Endpoint	Method	Behavior
<code>/api/auth/register</code>	POST	Creates user, hashes password, returns JWT + user object
<code>/api/auth/login</code>	POST	Validates credentials, resets roster/draft/record for fresh start, returns JWT
<code>/api/auth/me</code>	GET	Returns current user data (protected by auth middleware)

Design choice: Login resets the user's draft and roster every time, so each session is a fresh experience — users pick a new conference, draft a new team, and play a new season.

5.6 `server/routes/draft.js` — Draft Routes

Endpoint	Method	Behavior
<code>/api/draft/setup</code>	POST	Saves conference & league selection to the user document
<code>/api/draft/pool</code>	GET	Fetches 100 players from balldontlie, filters by conference, generates ratings
<code>/api/draft/pick</code>	POST	Adds a player to the user's roster (max 12, no duplicates)
<code>/api/draft/complete</code>	POST	Marks draft as done (requires minimum 5 players)

The pool endpoint fetches all NBA teams to determine conference membership, then fetches players and filters to only those on teams in the selected conference. It tries to use the paid `/season_averages` endpoint first; if that returns 401 (free tier), it silently falls back to profile-based ratings.

5.7 `server/routes/nba.js` — NBA API Proxy

Proxies requests to balldontlie.io, adding the API key. Endpoints:

- `GET /api/nba/teams` — All 30 NBA teams
- `GET /api/nba/players` — Paginated player search
- `GET /api/nba/players/:id/stats` — Season averages for one player (falls back to profile rating on free tier)
- `GET /api/nba/roster` — Full team roster with ratings

5.8 `server/routes/simulate.js` — Simulation Route

Accepts the user's team and opponent roster, runs the simulation engine, saves the result as a Game document, updates the user's W/L record, and returns the full result (scores, plays, box scores).

5.9 `server/services/playerRating.js` — Rating System

Contains two rating algorithms:

- `calculateRating(stats)` — Stats-based rating (40–99 scale). Requires season averages data. Weights: Scoring (max 35), Efficiency (max 20), Playmaking (max 15), Rebounding (max 10), Defense (max 10), Turnover penalty (max –5). Base floor of 40 for any NBA player.
- `calculateRatingFromProfile(player)` — Profile-based heuristic for the free API tier. Uses draft round, pick number, and years of experience to generate ratings. First-round lottery picks start at 72–88; undrafted players start at 55–67. Veterans get a bonus, players 14+ years decline slightly.

5.10 `server/services/simulation.js` — Game Engine

The core simulation engine models a full basketball game:

- **4 quarters × 12 minutes** (720 seconds each), with possessions consuming 5–24 seconds
- **Weighted player selection:** Higher-rated players get more touches via `pickWeightedPlayer()`
- **Shot outcomes** are probability-based: shot chance = 35% + (rating/99 × 25%), yielding 35–60% for any player
- **Play types:** 2-pointers, 3-pointers, free throws, missed shots with rebounds, assisted baskets, turnovers
- **Overtime:** If tied after 4 quarters, 5-minute OT periods repeat until a winner emerges

- **Output:** Returns team names, final scores, full play-by-play array (with quarter, clock, team, text, running scores), and player box scores (pts, reb, ast)

6. Frontend — Code Walkthrough

6.1 `client/src/App.jsx` — Routing & Navigation

Uses React Router v6 to define three routes:

Path	Component	Guard
<code>/login</code>	LoginPage	Redirects to <code>/draft</code> if already logged in
<code>/draft</code>	DraftPage	ProtectedRoute (requires auth token)
<code>/game</code>	GamePage	ProtectedRoute + DraftGate (requires completed draft)

A persistent nav bar shows the app name, username, W/L record, and a logout button. The `DraftGate` component redirects users to `/draft` if they haven't finished drafting.

6.2 `client/src/context/AuthContext.jsx` — Auth State Management

React Context provider that manages:

- `token` — JWT stored in localStorage for persistence across page reloads
- `user` — Full user object fetched from `GET /api/auth/me` on mount
- `login(token, user)` — Saves token and sets state
- `logout()` — Clears localStorage and resets state
- `loading` — Prevents flash of login page before token validation completes

6.3 `client/src/pages/LoginPage.jsx` — Authentication UI

A toggle form that switches between login and registration modes. Dark-themed card layout with orange accent branding. Calls `POST /api/auth/login` or `POST /api/auth/register` and passes the response to `AuthContext.login()`.

6.4 `client/src/pages/DraftPage.jsx` — Conference Selection & Draft

Two-phase page:

- Setup Phase:** Displays buttons for league selection (NBA, G-League, EuroLeague) and conference (Eastern/Western). Saves selection via `POST /api/draft/setup`.
- Draft Phase:** Two-panel layout — left panel shows the available player pool (100 players filtered by conference, sorted by rating), right panel shows the user's growing roster. Each player card displays name, position, team, and rating badge. Draft button

adds player to roster. "Complete Draft" button appears once minimum 5 players are picked.

6.5 `client/src/pages/GamePage.jsx` — Game Day

Displays the user's starting five, a grid of 30 NBA teams as selectable opponents, and the selected opponent's roster preview. Clicking "Simulate Game" sends both rosters to `POST /api/simulate` and passes the result to the GameCast component.

6.6 `client/src/components/GameCast.jsx` — Animated Play-by-Play

The centerpiece visual component. Features:

- **Live Scoreboard:** Shows team names, running scores, current quarter, and game clock
- **Play Stream:** Plays appear one by one with a CSS `fadeSlideIn` animation
- **Speed Controls:** Slow (1200ms), Normal (800ms), Fast (400ms), Turbo (100ms)
- **Playback Controls:** Play/Pause toggle, Skip to End button
- **Auto-scroll:** The play log auto-scrolls to keep the latest play visible

Uses `setInterval` with a ref for the timer, advancing `currentIndex` through the plays array at the selected speed.

7. API Endpoint Reference

Authentication

Method	Path	Auth	Body / Params	Response
POST	/api/auth/register	No	{ username, password }	{ token, user }
POST	/api/auth/login	No	{ username, password }	{ token, user } (roster reset)
GET	/api/auth/me	Yes	—	User object

Draft

Method	Path	Auth	Body / Params	Response
POST	/api/draft/setup	Yes	{ conference, league }	Confirmation
GET	/api/draft/pool	Yes	? season=2024&conference=East	Array of player objects with ratings
POST	/api/draft/pick	Yes	{ playerId, firstName, lastName, position, rating, stats }	Updated team
POST	/api/draft/complete	Yes	—	Confirmation

NBA Data (Proxy)

Method	Path	Auth	Params	Response
GET	/api/nba/teams	No	—	30 NBA teams
GET	/api/nba/players	No	?team_id, per_page, cursor	Paginated players
GET	/api/nba/players/:id/stats	No	?season	Rating + season averages

GET	/api/nba/roster	No	?team_id&season	Team roster with ratings
-----	-----------------	----	-----------------	--------------------------

Simulation & Games

Method	Path	Auth	Body	Response
POST	/api/simulate	Yes	{ opponentName, opponentPlayers[] }	Full game result (scores, plays, box scores)
GET	/api/games	No	—	Array of saved games
POST	/api/games/save	No	{ teamA, teamB, scoreA, scoreB, history }	Saved game document

8. How to Run Locally

Prerequisites

- Node.js 18 or higher
- MongoDB Community Server (or a MongoDB Atlas connection string)
- A free API key from app.balldontlie.io

Step 1 — Clone & Install

```
git clone https://github.com/zahyaa/NBASIM_Project_Full.git
cd NBASIM_Project_Full

# Install backend dependencies
cd server
npm install

# Install frontend dependencies
cd ../client
npm install
```

Step 2 — Start MongoDB

```
# macOS (Homebrew)
brew services start mongodb/brew/mongodb-community

# Linux (systemd)
sudo systemctl start mongod

# Or use MongoDB Atlas and set the connection string in .env
```

Step 3 — Configure Environment

```
cd server
cp .env.example .env
```

Edit `.env` with your values:

```
MONGODB_URI=mongodb://localhost:27017/nbasim
JWT_SECRET=your-secret-key-here
```



```
BALLDONTLIE_API_KEY=your-api-key-here
```

Step 4 — Start the Backend

```
cd server  
node server.js
```

Server starts on `http://localhost:5001`

Step 5 — Start the Frontend

```
cd client  
NODE_OPTIONS=--openssl-legacy-provider npm start
```

React dev server starts on `http://localhost:3000` and proxies API calls to port 5001.

Important Notes

- **Port 5001:** macOS AirPlay Receiver occupies port 5000. This project uses 5001 instead.
- **OpenSSL flag:** `NODE_OPTIONS=--openssl-legacy-provider` is required because react-scripts 3.x uses a legacy Webpack version incompatible with Node 18+'s OpenSSL 3.

9. How to Deploy to Production

Option A — Render (Recommended for simplicity)

1. Push code to GitHub (already done at `github.com/zahyaa/NBASIM_Project_Full`)
2. **Backend:** Create a "Web Service" on render.com
 - Root directory: `server`
 - Build command: `npm install`
 - Start command: `node server.js`
 - Set environment variables: `MONGODB_URI`, `JWT_SECRET`, `BALLDONTLIE_API_KEY`, `PORT=5001`
3. **Frontend:** Create a "Static Site" on Render
 - Root directory: `client`
 - Build command: `NODE_OPTIONS=--openssl-legacy-provider npm run build`
 - Publish directory: `client/build`
 - Add rewrite rule: `/* → /index.html` (for React Router)
4. Update the frontend's API base URL to point to the backend's Render URL (or set up a proxy)

Option B — Heroku

1. Add a root Procfile: `web: node server/server.js`
2. Serve the React build from Express by adding static file middleware:

```
app.use(express.static(path.join(__dirname, '../client/build')));
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, '../client/build/index.html'));
});
```

3. Build the client: `cd client && NODE_OPTIONS=--openssl-legacy-provider npm run build`
4. Set environment variables via Heroku dashboard or CLI
5. Deploy: `git push heroku main`

Option C — VPS (DigitalOcean, AWS EC2, etc.)

1. Install Node.js and MongoDB on the server
2. Clone the repo, install dependencies, build the client

3. Use `pm2` to run the server as a daemon: `pm2 start server/server.js --name basketball-sim`
4. Use Nginx as a reverse proxy:

```
server {  
    listen 80;  
    server_name yourdomain.com;  
  
    location /api/ {  
        proxy_pass http://localhost:5001;  
    }  
    location / {  
        root /path/to/client/build;  
        try_files $uri /index.html;  
    }  
}
```

5. Enable HTTPS with Let's Encrypt / Certbot

Database (Production)

Use **MongoDB Atlas** for a managed cloud database:

1. Create a free cluster at cloud.mongodb.com
2. Create a database user and whitelist your server's IP (or allow 0.0.0.0/0)
3. Copy the connection string and set it as `MONGODB_URI`

10. Environment Variables

Variable	Required	Description	Example
MONGODB_URI	Yes	MongoDB connection string	mongodb://localhost:27017/nbasim
JWT_SECRET	Yes	Secret key for signing JWT tokens	my-super-secret-key
BALLDONTLIE_API_KEY	Yes	API key from balldontlie.io	xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
PORT	No	Server port (default: 5001)	5001

— End of Document —

Basketball Simulator v1.0 | April 2026